

Relatório de Auditoria de Segurança

ControleBancario

Estado final, depois da remediação

Auditoria e remediação — 1 de setembro de 2026

Escopo auditado	Repositório ControleBancario na íntegra: as 8 apps Django (<code>`core`</code> , <code>`accounts`</code> , <code>`banking`</code> , <code>`bank_statements`</code> , <code>`transactions`</code> , <code>`management`</code> , <code>`reports`</code> , <code>`dashboard`</code>), todas as views e URLconfs, as camadas de serviço (≈5.000 linhas), os modelos e as 20 migrações, os 40+ templates, o JavaScript de <code>`static/js/`</code> , <code>`financeiro/settings.py`</code> , <code>`Dockerfile`</code> , <code>`compose.yaml`</code> , <code>`deploy/nginx/`</code> , os arquivos de ambiente (versionados e não versionados), o workflow de CI e o histórico completo do Git.
Linguagem	Python 3.13
Framework	Django 6 (8 apps, com camada de serviço separada das views)
ORM / query builder	Django ORM; migrações nativas
Banco	PostgreSQL
Mecanismo de auth	Django auth com <code>`AppUser`</code> próprio, mais DOIS backends — <code>`CaseInsensitiveUsernameBackend`</code> (autenticação) e <code>`AppPermissionBackend`</code> (autorização funcional contra o catálogo <code>`AppPermission`/`UserPermission`</code>); bloqueio de tentativas persistente em <code>`LoginLockout`</code>
Isolamento de dados	Mecanismo PRÓPRIO e real: <code>`UserOwnerAccess`</code> concede view/create/update/delete por TITULAR (<code>`AccountOwner`</code>), e as contas herdam o escopo do titular (<code>`banking.services.accessible_account_ids`</code>)
Frontend	Templates Django + HTMX (vendORIZADO) e Chart.js; CSP estrita por middleware próprio, sem <code>`unsafe-inline`</code>
Deploy	Docker Compose (gunicorn, WhiteNoise com manifesto), Nginx com TLS no VPS Oracle

Nota metodológica

Este é o único dos quatro aplicativos com isolamento de dados de verdade — ``UserOwnerAccess``, por titular, com quatro verbos. Por isso a categoria 1 pôde ser auditada na sua forma plena: para CADA consulta de listagem, busca, agregação, relatório e exportação, foi verificado se ela passa por ``accessible_owner_ids`` ou ``accessible_account_ids`` antes de tocar o banco. A categoria 3 (IDOR) foi auditada percorrendo TODOS os handlers que recebem ID, em todas as 8 apps, e não uma amostra — foi essa varredura exaustiva que produziu o achado CB-01, num serviço que é a única exceção num conjunto de dezenas de funções corretas.

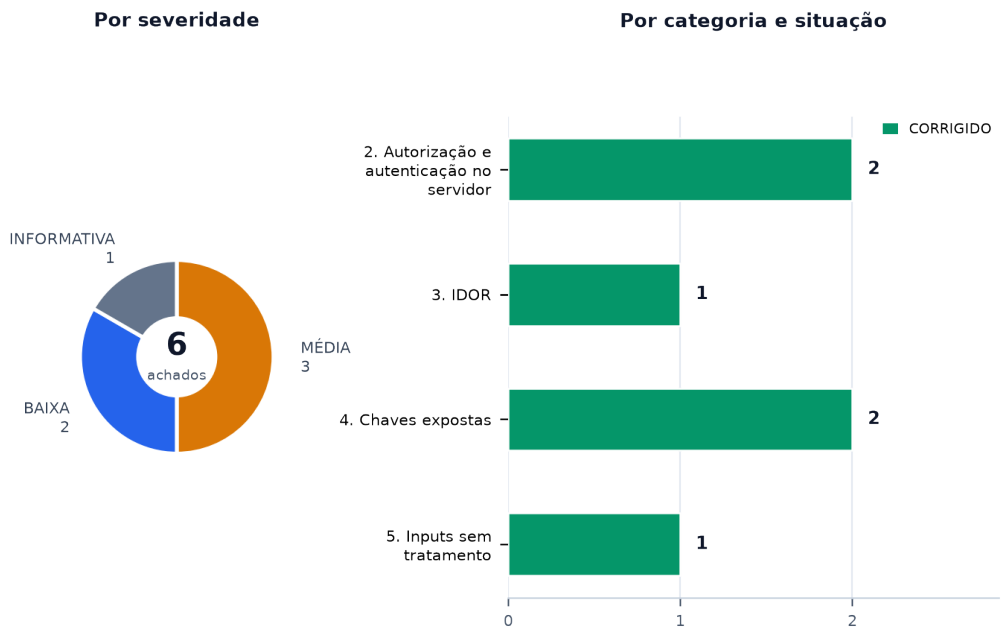
Categoria pedida	Como foi mapeada nesta stack
1. Banco sem tranca (isolamento de inquilino/dono)	O mecanismo é `UserOwnerAccess` (por titular) mais a herança de escopo das contas. Auditadas todas as consultas que produzem lista, agregado, relatório ou exportação, em `reports/services.py` (1.615 linhas), `transactions/services.py` (1.631), `bank_statements/`, `banking/`, `management/` e `dashboard/`, confirmando a passagem por `accessible_owner_ids`/`accessible_account_ids`.
2. Permissão definida no navegador	Cruzamento de cada `can_*` passado ao template com o decorator `@permission_required` da rota correspondente, e varredura da URLconf inteira pelo atributo `permissao_exigida` que o decorator grava na view. Auditado também o caminho de escalonamento: quem tem `permissions.manage` consegue elevar a si mesmo?
3. IDOR	Percorridos TODOS os handlers das 8 apps que recebem ID em caminho, query ou corpo — lançamento, anexo, linha de extrato, lote de importação, conta, titular, categoria, tag, projeto, orçamento e usuário —, verificando em cada um a existência da checagem de escopo por titular.
4. Chaves expostas (hardcode)	Varredura de código, templates, `settings.py`, `compose.yaml`, `Dockerfile`, scripts PowerShell, workflow de CI, documentação, arquivos `.env` (inclusive os NÃO versionados) e do histórico completo do Git. Atenção específica aos defaults `\${VAR:-valor}` do Compose e à ausência de validação de boot que os recuse.
5. Inputs sem tratamento (XSS)	Frontend: varredura de `static/js/` por `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `eval`, `new Function` e `document.write`. Backend: busca por ` safe`, `mark_safe`, `format_html` e `{% autoescape off %}` em todos os templates e templatetags — com leitura de cada `mark_safe` encontrado para verificar a procedência do que ele marca.

1. Resumo executivo

Terceira leitura do mesmo relatório, depois de duas rodadas de remediação. Das seis constatações da auditoria, as três de severidade média foram corrigidas na primeira rodada (1 de setembro) e as três restantes — duas baixas e uma informativa — foram fechadas nesta segunda rodada. Nenhuma delas era defeito de lógica: ``.env.docker`` tinha os dois segredos operacionais duplicados em texto claro (já provisionados em ``.secrets/``) e ``.scripts/package_clean_zip.py`` não excluía ``.secrets/`` nem ``.certs/`` do ZIP de distribuição; o default de ``.POSTGRES_USER`` caía no superusuário ``.postgres`` do cluster quando a variável faltasse; e a templatetag ``.icon`` marcava um rótulo livre como seguro sem escapá-lo. As seis constatações da auditoria estão corrigidas e verificadas por teste. A suíte de qualidade do projeto ficou verde: Ruff sem apontamento e 265 testes.



Achados por severidade e por categoria



2. Estado da remediação

As seis constatações da auditoria estão corrigidas em duas rodadas. As três primeiras (1 de setembro) foram feitas em árvore de trabalho, sem commit; as três desta rodada seguem o mesmo regime — nada commitado, cada uma com o teste que a fixa. Duas decisões de projeto continuam merecendo registro porque não são óbvias no diff, mais uma nova desta rodada.

- O registro de auditoria da remoção de orçamento ficou no SERVIÇO, e não na view como nas demais ações de Gestão: `Model.delete()` do Django zera a chave primária do objeto em memória, então o identificador precisa ser lido antes da exclusão.
- Na tela de Permissões, esconder os botões foi consequência e não solução: a trava que vale é a verificação no servidor.
- A limpeza de `.env.docker` (CB-03) é a única das seis correções que não aparece no diff do Git: o arquivo é local e ignorado de propósito. A evidência de que ela aconteceu é o próprio conteúdo do arquivo hoje, não um commit.

Achado	Situação	O que mudou, e como foi verificado
CB-01 MÉDIA	CORRIGIDO	<code>`retire_budget`</code> apaga orçamento de qualquer titular, sem checar escopo <code>`retire_budget`</code> passou a receber <code>`user`</code> e a exigir <code>`accessible_owner_ids(user, 'delete')`</code> sobre <code>`budget.owner_id`</code>, no mesmo formato que <code>`save_budget`</code> já usava 25 linhas acima (<code>management/services.py:225-248</code>); a view repassa <code>`request.user`</code> (<code>management/views.py:122</code>). O orçamento de titular fora do escopo passa a responder a mesma mensagem de inexistente, sem revelar que o registro existe. A auditoria que faltava foi acrescentada em <code>services.py:283</code>. Verificação: Dois testes novos em <code>tests/test_management_lifecycle.py:86</code> e <code>:99</code> — o caminho legítimo com registro de auditoria, e o orçamento de outro titular, que permanece no banco.
CB-02 MÉDIA	CORRIGIDO	<code>`permissions.manage`</code> permite auto-concessão de todo o escopo de dados As três ações que concedem privilégio na tela de Permissões passaram a exigir <code>`user_type == administrator`</code> (<code>core/views.py:229</code>); <code>`permissions.manage`</code> virou permissão de leitura da tela. A interface deixou de oferecer os três botões de gravação a quem não é administrador (<code>`pode_conceder`</code>, <code>core/views.py:329</code> e <code>templates/permissions/index.html:79, 214 e 297</code>) — mas quem recusa é o servidor. Verificação: <code>tests/test_authorization.py:98</code>, parametrizado nas três ações; <code>:115</code> garante que o administrador continua concedendo; <code>:132</code> confere que os botões somem.
CB-04 MÉDIA	CORRIGIDO	<code>`/admin/`</code> do Django dá acesso real ao modelo de usuário e contorna o bloqueio de tentativas <code>`django.contrib.admin`</code> saiu de <code>`INSTALLED_APPS`</code>, com o motivo registrado no lugar dele (<code>financeiro/settings.py:61-75</code>), e <code>`path('admin/', admin.site.urls)`</code> saiu da <code>URLconf</code>; os oito <code>`admin.py`</code> vazios do <code>`startapp`</code> foram apagados, para que ninguém leia neles a promessa de um painel que não existe. Depois disso o mantenedor zerou <code>`is_staff`</code> e <code>`is_superuser`</code> da conta <code>`Admin`</code> (id 5) no banco de produção, preservando <code>`is_active`</code> — a conta continua existindo e a regra do último administrador segue valendo. Verificação: <code>tests/test_rotas_orfas_removidas.py:54</code> e <code>:64</code> — <code>`/admin/`</code> e <code>`/admin/login/`</code> respondem 404, e não 302: um 302 significaria que a <code>URLconf</code> ainda casa o caminho. <code>tests/test_permissoes_por_rota.py</code> deixou de abrir exceção para <code>`admin/`</code> e voltou a varrer a <code>URLconf</code> inteira.

Achado	Situação	O que mudou, e como foi verificado
CB-03 BAIXA	CORRIGIDO	Segredos duplicados em texto claro em <code>.env.docker</code> <code>.env.docker</code> teve <code>POSTGRES_PASSWORD</code> e <code>DJANGO_SECRET_KEY</code> esvaziados — os dois já estavam em <code>.secrets/</code>, confirmados por hash, e o arquivo é local e não versionado (nunca passa por commit). <code>scripts/package_clean_zip.py</code> ganhou <code>.certs</code> e <code>.secrets</code> em <code>EXCLUDED_DIRS</code>: antes desta correção, um ZIP "limpo" gerado pelo script incluiria os dois diretórios de segredo por inteiro — <code>.gitignore</code> e <code>.dropboxignore</code> não cobrem esse caminho, que mantém a própria lista de exclusão, independente das outras duas. Verificação: <code>tests/test_package_clean_zip.py</code> (novo): <code>test_secrets_directory_e_excluida</code>, <code>test_certs_directory_e_excluida</code>, <code>test_env_docker_continua_excluido</code> (regressão) e <code>test_arquivo_comum_do_projeto_nao_e_excluido</code> (sanidade, contra exclusão ampla demais).
CB-05 BAIXA	CORRIGIDO	<code>POSTGRES_USER</code> cai no superusuário <code>postgres</code> quando a variável não é informada O default de <code>POSTGRES_USER</code> nos três pontos do <code>compose.yml</code> (linhas 22, 78, 88) mudou de <code>postgres</code> para <code>controle_bancario</code>. <code>financeiro/settings.py</code> ganhou uma checagem de boot logo depois de montar <code>DATABASES</code>: se <code>DATABASES['default']['USER'] == 'postgres'</code>, levanta <code>RuntimeError</code> nomeando a variável — mesma família de validação que <code>USE_HTTPS</code> e <code>CSRF_TRUSTED_ORIGINS</code> já tinha. <code>.env.docker(.example)</code> e <code>.env.vps.example</code> já declaravam <code>POSTGRES_USER=controle_bancario</code> explicitamente, então nenhuma implantação real muda de comportamento — o que fecha é o default silencioso, não a configuração em uso. Verificação: <code>tests/test_operational_configuration.py</code> (novos): <code>test_settings_recusa_conectar_como_postgres</code>, <code>test_settings_aceita_postgres_user_dedicado</code>, <code>test_default_de_postgres_user_nao_e_o_superuser</code> (importa <code>financeiro.settings</code> sem a variável definida e confere que <code>DATABASES['default']['USER'] == 'controle_bancario'</code>) e <code>test_compose_nao_usa_postgres_como_padrao_de_postgres_user</code>.
CB-06 INFORMATIVA	CORRIGIDO	A <code>templatetag</code> <code>icon</code> marca o rótulo como seguro sem escapá-lo <code>icon(name, label=)</code> em <code>core/templatetags/icons.py</code> passou a escapar <code>label</code> com <code>django.utils.html.escape</code> antes de interpolar no <code></code>, dentro do próprio <code>mark_safe</code> — fecha a categoria no arquivo, sem depender de nenhum call site futuro lembrar de escapar antes de passar o segundo parâmetro. Verificação: <code>tests/test_icons_templatetag.py</code> (novo): <code>test_label_com_html_e_escalado</code> (prova que <code><script></code> vira <code>&lt;script></code>), <code>test_label_comum_continua_visivel</code> (regressão do caminho feliz), <code>test_sem_label_nao_acrescenta_span</code> e <code>test_nome_desconhecido_devolve_vazio</code>.

3. Pontos fortes e pontos fracos

3.1 O que está protegido (com evidência)

Controle verificado	Evidência no código
Escopo por titular herdado corretamente pelas contas `accessible_account_ids` deriva de `accessible_owner_ids`, e `can_access_account` resolve o titular da conta antes de decidir. Esse é o helper central, e é ele que os módulos de Bancos, Transações e Cadastros usam — não uma cópia por app.	banking/services.py:76-98 e accounts/services.py:546-568
Choke point dos relatórios valida o filtro vindo da URL `selected_context` recusa `owner_id` fora dos permitidos e resolve `account_id` com `filter(pk=..., owner_id__in=allowed_owner_ids)`. Um ID de outro titular na query string é descartado — e ainda é ANOTADO na requisição, para o middleware limpar a barra de endereços e o filtro morto não voltar num F5 ou num favorito.	reports/services.py:209-233
Planejamento anual faz interseção, não confiança `owner_ids` e `account_ids` vindos da query passam por `allowed_owner_ids.intersection(...)` e `allowed_account_ids.intersection(...)`. Pedir o titular de outra pessoa não devolve erro nem dado: devolve conjunto vazio.	reports/views.py:190-210
Conciliação bancária escopada em todos os pontos `_get_line_in_scope` resolve a linha e confere `can_access_account`; `reconcile_line_with_entry` confere o lançamento SEPARADAMENTE, com ação `update`, e ainda exige que linha e lançamento pertençam à mesma conta. As listagens de pendentes e conciliadas filtram por `accessible_account_ids`.	bank_statements/reconciliation.py:158-168, 215-241, 46-66
Download de anexo verifica o escopo do lançamento, não só a permissão funcional `attachment_for_download` resolve o anexo por ID e devolve `None` quando `can_access_entry` recusa — a view trata `None` como 'não encontrado'. Não há caminho para baixar comprovante de conta fora do escopo.	bank_statements/attachments.py:156-165 e 86-87
Transferência interna com autorização resolvida pela ponta de origem Uma transferência pode sair de uma conta acessível para uma de outro titular. `_authorization_entry` resolve a autorização sempre pela ponta de origem, de modo que a contraparte é exibida e atualizada junto sem que isso conceda acesso geral à conta de destino.	transactions/access.py:18-32
Autorização funcional declarativa, com varredura da URLconf que a cobra O decorator grava `permissao_exigida` na view, e existe um teste que percorre a URLconf lendo esse atributo. O comentário registra o motivo: foi assim que quatro telas ficaram com chave no catálogo e nenhuma verificação, por tempo indeterminado.	core/permissions.py:47-52 e tests/test_permissoes_por_rota.py
Escopo de DADOS deliberadamente separado da permissão FUNCIONAL `_has_broad_owner_access` dá visão ampla a administrador e super usuário, mas isso NÃO lhes concede o direito de gerir permissões — a distinção está escrita no docstring e implementada em duas funções diferentes.	accounts/services.py:535-544 e 102-118
Confirmação de alteração 'este e os próximos' assinada pelo servidor O token vem de `django.core.signing.dumps`, é vinculado ao ID do lançamento e ao escopo, e expira em 10 minutos. Um POST direto sem a confirmação concluída é recusado pelo service — a confirmação não depende do JavaScript.	transactions/services.py:1038-1060

Controle verificado	Evidência no código
Upload de anexo e de extrato com nome saneado e assinatura conferida `_sanitize_filename` remove separador de caminho e reduz a um allowlist, então um nome enviado não escapa do diretório de destino; e `_validate_attachment_signature` confere os bytes iniciais contra a extensão. Tetos de 5 MB (extrato), 5.000 linhas e 10 MB (anexo) vêm de settings.	bank_statements/services.py:27-36 e attachments.py:52-71
Nenhum XSS: zero `!safe`, e cada `mark_safe` marca constante literal Varredura completa dos templates e templatetags: nenhum `!safe`, nenhum `{% autoescape off %}`. Os quatro `mark_safe` foram lidos um a um — dois em `icons.py` (SVG de um dicionário fechado), um em `navegacao.py` (constante literal, com `noqa` explicando) e um em `sharedauth_tags.py` (o ícone da biblioteca). O único dado do cliente vai por `json_script`, que não é executável.	core/templatetags/*.py e templates/dashboard/_content.html:58
CSP estrita, aplicada em desenvolvimento e produção Middleware próprio sobre os valores do SharedAuth, sem nonce e sem `unsafe-inline`, com o raciocínio registrado: um nonce exigiria ` <style>` embutido, o HTMX o perderia na troca de tela, e expor o nonce num `<meta>` entregaria ao DOM justamente o segredo que ele é.</td><td>core/security.py:72-105</td></tr><tr><td>`Vary` correto para HTMX: `HX-Request` E `HX-Target` A mesma URL responde página inteira ou fragmento conforme o ALVO da troca, não só conforme o cabeçalho. Declarar apenas `HX-Request` deixaria um cache servir o fragmento a quem pediu a página. O comentário registra que a recomendação original da auditoria anterior ficou desatualizada com a migração para HTMX.</td><td>core/security.py:28-46</td></tr><tr><td>Segredo obrigatório e `DEBUG` desligado por padrão `_read_required_secret` levanta se o segredo não existe, e `REQUIRE_FILE_SECRETS=true` no Compose recusa a forma direta por variável. `DEBUG` só liga quando pedido explicitamente. `USE_HTTPS=True` exige `CSRF_TRUSTED_ORIGINS` não vazio e só com origens HTTPS, ou a aplicação não sobe.</td><td>financeiro/settings.py:17-44 e 240-247</td></tr><tr><td>Bloqueio de tentativas de login persistente, por conta e origem `LoginLockout` guarda contagem e prazo no banco (não em memória do processo), chaveado por usuário normalizado mais endereço. Sobrevive a reinício e vale entre os workers do gunicorn.</td><td>accounts/login_lockout.py:27-77 e accounts/views.py:107-131</td></tr><tr><td>Log de autenticação sanitizado contra forja de linha O nome de usuário digitado passa por `sharedauth.logs.sanitarizar_log` antes de ir ao log, com o comentário explicando que `.strip()` tira espaço das pontas mas não a quebra de linha do meio — e é a do meio que forja um registro falso indistinguível de um verdadeiro.</td><td>accounts/auth_backends.py:37-60</td></tr><tr><td>Sem segredo no histórico do Git Nenhum arquivo `.env`, `.pem` ou de segredo jamais foi adicionado ao repositório. O único arquivo com 'secret' no nome é o script de provisionamento, que gera valor em vez de contê-lo. O PAT do build entra por BuildKit secret com `umask 077`.</td><td>git log --all --diff-filter=A e .github/workflows/ci.yml:45-47</td></tr><tr><td>Escopo por titular também na remoção de orçamento A única exceção que a auditoria encontrou no isolamento por titular foi fechada: a remoção confere `accessible_owner_ids(user, 'delete')` antes de tocar o registro, e o titular fora do escopo recebe a resposta de inexistente.</td><td>management/services.py:248 · tests/test_management_lifecycle.py:99</td></tr><tr><td>Concessão de privilégio restrita a `administrator` As três ações que concedem perfil, permissão funcional e acesso por titular exigem o tipo de usuário, e não apenas a permissão funcional `permissions.manage` — que ficou sendo permissão de leitura da tela.</td><td>core/views.py:229 · tests/test_authorization.py:98</td></tr></table> Auditoria de Segurança (estado final) — ControleBancario Página 7 </style>	

Controle verificado	Evidência no código
Sem painel administrativo paralelo Não existe mais um segundo caminho de administração de contas por fora de <code>`user_mutation_block_message`</code> , da regra do último administrador e do <code>`log_audit_event`</code> , nem uma rota que verifique senha sem passar pelo <code>`LoginLockout`</code> .	<code>financeiro/settings.py:61-75 · tests/test_rotas_orfas_removidas.py:64</code>
As três correções de configuração fecham default silencioso, não configuração em uso Em nenhum dos três casos (CB-03, CB-05, CB-06) a implantação real precisou mudar: <code>`.env.docker.example`</code> e <code>`.env.vps.example`</code> já declaravam <code>`POSTGRES_USER=controle_bancario`</code> explicitamente, e nenhum template chama <code>`{% icon %}`</code> com um segundo argumento hoje. As três correções fecham o que aconteceria se alguém esquecesse a variável ou escrevesse um call site novo — não um incidente em curso.	<code>compose.yaml:22,78,88; financeiro/settings.py (checagem de boot logo após DATABASES); core/templatetags/icons.py:38-48; scripts/package_clean_zip.py (EXCLUDED_DIRS)</code>

3.2 Os riscos centrais

- O isolamento por titular é aplicado dezenas de vezes e falha em um ponto: ``retire_budget`` arquiva ou apaga um orçamento mensal pelo ID, sem receber o usuário e sem consultar ``accessible_owner_ids`` — enquanto ``save_budget``, na mesma camada, faz exatamente essa checagem. É a assimetria que caracteriza um IDOR real, não uma decisão de projeto. FECHADO em 01/09: a função passou a receber o usuário e a conferir ``accessible_owner_ids``, com dois testes fixando o comportamento.
- A trava contra escalonamento existe para o TIPO de usuário e não para o ESCOPO de dados. ``user_mutation_block_message`` impede um não-administrador de promover alguém a ``super_user``; nada impede que a mesma pessoa, tendo ``permissions.manage``, marque para si todas as permissões funcionais e todos os titulares na matriz de acesso. FECHADO em 01/09: as três ações de concessão passaram a exigir ``administrator``, e ``permissions.manage`` virou permissão de leitura.
- O ``/admin/`` do Django está habilitado e roteado, e a conta ``Admin`` tem ``is_staff`` e ``is_superuser`` em produção — verificado no banco. ``django.contrib.auth.admin`` registra o modelo de usuário sozinho, então esse caminho edita contas, senhas e papéis por fora de ``user_mutation_block_message``, da regra do último administrador e da trilha de auditoria. E ``/admin/login/`` verifica senha sem passar por ``AppLoginView``, portanto sem o ``LoginLockout``. FECHADO em 01/09: o app do admin e a rota foram removidos, e as marcas ``is_staff`` e ``is_superuser`` da conta ``Admin`` foram zeradas em produção.
- ``.env.docker`` guarda a senha do PostgreSQL e a ``DJANGO_SECRET_KEY`` em texto claro, com os mesmos valores que já estão em ``.secrets/`` — duplicação que é o desenho do script de provisionamento, não um acidente. Não houve exposição, e o ``REQUIRE_FILE_SECRETS`` faz o contêiner recusar a forma direta; o que sobra é superfície. FECHADO nesta rodada: os dois valores foram esvaziados do arquivo, e ``package_clean_zip.py`` passou a excluir ``.secrets/`` e ``.certs/`` do ZIP de distribuição.

4. Achados detalhados

Um bloco por categoria. Todo achado abaixo foi conferido no código real: arquivo, linha e trecho vêm da árvore auditada, não de inferência. O diagnóstico está preservado exatamente como foi escrito na auditoria — inclusive nos achados já corrigidos, cujo trecho de código citado não existe mais na árvore. O que mudou vem depois dele, no bloco de correção.

2. Autorização e autenticação no servidor

Severidade	Arquivo:linha	Descrição
MÉDIA	core/views.py:102-105, 211-244 + accounts/ser	[CB-02] `permissions.manage` permite auto-concessão de todo o escopo de dados A tela de Permissões exige apenas `permissions.manage`. O usuário-alvo vem de `request.GET/POST['user_id']`, resolvido sobre `list_manageable_users()`, que devolve TODOS os usuários — inclusive quem está operando. Duas ações da mesma tela não têm trava de auto-elevação: `save_function_permissions` grava o conjunto inteiro de permissões funcionais no alvo, e `save_owner_access` grava a matriz completa de view/create/update/delete sobre TODOS os titulares. Por que é explorável: A única proteção contra manipular a própria conta é o oposto do que seria preciso: ela impede a pessoa de REMOVER a própria `permissions.manage`, e não de ACRESCENTAR o resto. Um usuário de tipo `user`, com escopo restrito a um titular e com `permissions.manage` concedida, seleciona a si mesmo, marca todas as caixas de permissão funcional e todas as linhas da matriz de titulares, salva — e passa a ver e alterar todo o dado financeiro do sistema. O contraste mostra que a preocupação com escalonamento existe e parou no lugar errado: `user_mutation_block_message` (accounts/services.py:356-360) bloqueia explicitamente 'somente administrador pode criar ou promover usuários privilegiados', cobrindo o `user_type` — e nada equivalente sobre o escopo de dados. Impacto: Escalonamento de privilégio horizontal e vertical dentro do modelo de permissões: acesso completo aos dados de todos os titulares e a todas as funções, sem passar por `tables.users.manage` nem virar `administrator`. A ação FICA registrada na trilha de auditoria (`log_audit_event` é chamado nas duas), o que ajuda a detectar, mas não impede. Condição de explorabilidade: Exige que `permissions.manage` tenha sido concedida a uma conta não administradora. Se ela só existe em contas `administrator`, não há escalonamento — o administrador já tem tudo por regra do sistema (`has_function_permission` devolve `True` para esse tipo). O risco nasce no momento em que alguém delega a gestão de permissões sem delegar o resto, que é exatamente o caso de uso que a permissão existe para atender. <pre> selected_user = users.filter(id=selected_user_id).first() if selected_user_id else users.first() ... if selected_user.id == request.user.id and 'permissions.manage' not in allowed_keys: allowed_keys.add('permissions.manage') messages.warning(request, "Protecao aplicada: voce nao pode remover o proprio acesso...") </pre> CORRIGIDO — o que mudou: As três ações que concedem privilégio na tela de Permissões passaram a exigir `user_type == administrator` (core/views.py:229); `permissions.manage` virou permissão de leitura da tela. A interface deixou de oferecer os três botões de gravação a quem não é administrador (`pode_conceder`, core/views.py:329 e templates/permissions/index.html:79, 214 e 297) — mas quem recusa é o servidor.
CORRIGIDO	vices.py:499-527	

Severidade	Arquivo:linha	Descrição
		Verificação: tests/test_authorization.py:98, parametrizado nas três ações; :115 garante que o administrador continua concedendo; :132 confere que os botões somem.
MÉDIA CORRIGIDO	financeiro/settings.py :61 + financeiro/urls.py:76 + accounts/views.py:107-114	[CB-04] `/admin/` do Django dá acesso real ao modelo de usuário e contorna o bloqueio de tentativas `django.contrib.admin` está em `INSTALLED_APPS` e `path('admin/', admin.site.urls)` está roteado. Nenhum `admin.py` do projeto registra modelo — os oito são stubs vazios do `startapp` —, mas `django.contrib.auth.admin` registra o modelo de usuário SOZINHO, e o modelo registrado é o substituído (`AppUser`). E o bloqueio de tentativas de login existe apenas em `AppLoginView.post`, que serve `/login`. Por que é explorável: São duas coisas, e a consulta ao banco de produção em 01/09/2026 confirmou que ambas estão ao alcance: a conta `Admin` tem `is_staff=True` e `is_superuser=True`. PRIMEIRA — com essa conta, `/admin/` edita `AppUser` diretamente: criar conta, trocar senha, alterar `user_type`, ligar `is_superuser`. Tudo por fora de `user_mutation_block_message` (que impede promover usuário privilegiado sem ser administrador), por fora da regra do último administrador, por fora da obrigação de trocar senha temporária e — o que mais importa numa investigação — SEM registrar nada em `log_audit_event`. SEGUNDA — `AdminAuthenticationForm` chama `authenticate()` antes de recusar por falta de `is_staff`, então `/admin/login/` verifica a senha de QUALQUER conta sem passar por `assert_login_not_throttled` e sem chamar `register_failed_login_attempt`. O `LoginLockout`, principal controle antiforça-bruta do sistema e o único que sobrevive a reinício, fica contornado por essa rota. Impacto: Um caminho paralelo de administração de contas, sem as travas da aplicação e sem trilha de auditoria, disponível a quem obtiver a senha da conta `Admin`. Somado a isso, adivinhação de senha ilimitada contra qualquer conta conhecida, sem acionar o bloqueio e sem deixar o rastro que `register_failed_login_attempt` deixaria. O sistema tem hoje quatro contas, duas delas do tipo `user` — não é um cenário de usuário único. Condição de explorabilidade: Verificado no banco de produção em 01/09/2026: `is_staff=True` e `is_superuser=True` na conta `Admin` (último login em 30/08). A conta de trabalho do mantenedor é `mspa`, que NÃO tem essas marcas. O caminho `/admin/` é o padrão do Django e não precisa ser descoberto; o Nginx faz proxy dele junto com o resto. <pre> class AppLoginView(LoginView): def post(self, request, *args, **kwargs): remote_addr = request.META.get('REMOTE_ADDR') try: assert_login_not_throttled(request.POST.get('username'), remote_addr) except LoginThrottledError as exc: ... </pre> CORRIGIDO — o que mudou: `django.contrib.admin` saiu de `INSTALLED_APPS`, com o motivo registrado no lugar dele (financeiro/settings.py:61-75), e `path('admin/', admin.site.urls)` saiu da URLconf; os oito `admin.py` vazios do `startapp` foram apagados, para que ninguém leia neles a promessa de um painel que não existe. Depois disso o mantenedor zerou `is_staff` e `is_superuser` da conta `Admin` (id 5) no banco de produção, preservando `is_active` — a conta continua existindo e a regra do último administrador segue valendo. Verificação: tests/test_rotas_orfas_removidas.py:54 e :64 — `/admin/` e `/admin/login/` respondem 404, e não 302: um 302 significaria que a URLconf ainda casa o caminho. tests/test_permissoes_por_rota.py deixou de abrir exceção para `admin/` e voltou a varrer a URLconf inteira.

3. IDOR

Severidade	Arquivo:linha	Descrição
MÉDIA	management/services.py:225-244 + management/views.py:117-122	[CB-01] `retire_budget` apaga orçamento de qualquer titular, sem checar escopo `retire_budget(budget_id)` recebe apenas o ID: não recebe o usuário e não consulta `accessible_owner_ids`. Ela resolve o `MonthlyBudget`, decide entre arquivar (se há lançamento realizado no mês/categoria/titular) ou apagar de vez, e executa. A view que a chama exige `management.manage` — a permissão FUNCIONAL — e nada mais. Por que é explorável: A assimetria é a prova. `save_budget`, na MESMA camada e a 25 linhas de distância, faz a checagem correta: `if owner_id_int not in set(accessible_owner_ids(user, "update")): raise ValueError("Acesso negado...")`. E a listagem da própria tela também está escopada — `management_view` monta `budget_rows_for_period` a partir de `options.owners`, que sai de `context_options` e já veio filtrado. Ou seja: a pessoa não VÊ o orçamento de outro titular na tela, não pode CRIAR nem EDITAR orçamento de outro titular, e pode APAGAR o de qualquer um. Basta um POST direto: `POST /management/budget/retire/` com `budget_id=<id de outro titular>` e o token CSRF da própria sessão. Impacto: Escrita destrutiva atravessando a fronteira de titular, que é o mecanismo de isolamento deste sistema. O dado perdido é de planejamento (valor orçado por categoria/mês), não do razão — o que limita o estrago —, mas a exclusão é definitiva quando não há lançamento realizado no período, e não há tela que a desfça. O evento também não entra na trilha de auditoria: `retire_budget_view` não chama `log_audit_event`. Condição de explorabilidade: Exige uma conta autenticada com a permissão funcional `management.manage` e que exista mais de um titular no sistema, com acessos distintos. Administradores e super usuários têm acesso amplo por regra (`_has_broad_owner_access`), então para eles o comportamento é o mesmo com ou sem o defeito — o alvo é a conta de tipo `user` com escopo restrito. <pre>def retire_budget(budget_id) -> tuple[MonthlyBudget, str]: try: budget = MonthlyBudget.objects.select_related("owner", "category").get(id=int(budget_id)) except (MonthlyBudget.DoesNotExist, TypeError, ValueError): raise ValueError("Orçamento nao encontrado.") from None ... budget.delete() return budget, "deleted"</pre> CORRIGIDO — o que mudou: `retire_budget` passou a receber `user` e a exigir `accessible_owner_ids(user, 'delete')` sobre `budget.owner_id`, no mesmo formato que `save_budget` já usava 25 linhas acima (management/services.py:225-248); a view repassa `request.user` (management/views.py:122). O orçamento de titular fora do escopo passa a responder a mesma mensagem de inexistente, sem revelar que o registro existe. A auditoria que faltava foi acrescentada em services.py:283. Verificação: Dois testes novos em tests/test_management_lifecycle.py:86 e :99 — o caminho legítimo com registro de auditoria, e o orçamento de outro titular, que permanece no banco.

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
BAIXA CORRIGIDO	.env.docker:3, 6 (não versionado)	[CB-03] Segredos duplicados em texto claro em <code>.env.docker</code> O arquivo <code>.env.docker</code> guarda <code>POSTGRES_PASSWORD=<redacted></code> Por que é explorável: Não é exposição, e vale registrar o que foi verificado: o <code>.gitignore</code> recusa o arquivo (confirmado por <code>git check-ignore</code>); o histórico completo do Git não tem nenhum arquivo de segredo; o <code>.dropboxignore</code> já excluía <code>.env</code> da sincronização de nuvem ANTES desta auditoria (conferido em <code>git show HEAD:.dropboxignore</code>); e a pasta <code>.secrets/</code> foi verificada em dropbox.com, inclusive na lixeira — nunca subiu. Este projeto tem ainda a mitigação mais forte dos quatro: o <code>compose.yaml</code> liga <code>REQUIRE_FILE_SECRETS: "true"</code>, e <code>_read_required_secret</code> (<code>financeiro/settings.py:29-34</code>) então RECUSA a forma direta por variável. O contêiner operacional não consegue usar esses valores nem por engano. O que resta é superfície: o mesmo segredo em dois arquivos, e a <code>DJANGO_SECRET_KEY</code> assina três coisas neste projeto — cookie de sessão, token CSRF e o token de confirmação de 'este e os próximos' de <code>transactions/services.py</code>. Impacto: Nenhum comprometimento conhecido. O custo é de higiene: cada cópia adicional é mais um lugar de onde o segredo pode escapar por um caminho não previsto — e este repositório tem um <code>scripts/package_clean_zip.py</code>, que é exatamente o tipo de ferramenta em que um arquivo a mais na raiz costuma entrar sem ninguém notar. Condição de explorabilidade: <code>REQUIRE_FILE_SECRETS=true</code> no <code>compose.yaml</code> faz o contêiner recusar a forma direta. A cópia do <code>.env.docker</code> só vale para o script de provisionamento e para comandos locais fora do Compose. <code>POSTGRES_PASSWORD=<redacted></code> CORRIGIDO — o que mudou: <code>.env.docker</code> teve <code>POSTGRES_PASSWORD</code> e <code>DJANGO_SECRET_KEY</code> esvaziados — os dois já estavam em <code>.secrets/</code>, confirmados por hash, e o arquivo é local e não versionado (nunca passa por commit). <code>scripts/package_clean_zip.py</code> ganhou <code>.certs</code> e <code>.secrets</code> em <code>EXCLUDED_DIRS</code>: antes desta correção, um ZIP "limpo" gerado pelo script incluiria os dois diretórios de segredo por inteiro — <code>.gitignore</code> e <code>.dropboxignore</code> não cobrem esse caminho, que mantém a própria lista de exclusão, independente das outras duas. Verificação: <code>tests/test_package_clean_zip.py</code> (novo): <code>test_secrets_directory_e_excluida</code>, <code>test_certs_directory_e_excluida</code>, <code>test_env_docker_continua_excluido</code> (regressão) e <code>test_arquivo_comum_do_projeto_nao_e_excluido</code> (sanidade, contra exclusão ampla demais).
Severidade	Arquivo:linha	Descrição
BAIXA CORRIGIDO	compose.yaml:22, 78, 88	[CB-05] <code>POSTGRES_USER</code> cai no superusuário <code>postgres</code> quando a variável não é informada Os três pontos do Compose que nomeiam o usuário do banco usam <code>`\${POSTGRES_USER:-postgres}`</code>. Ausente a variável, o serviço do PostgreSQL é inicializado com o papel <code>postgres</code> — o superusuário do cluster — e a aplicação conecta com ele. Por que é explorável: O default escolhido é o mais privilegiado possível, e não o mínimo necessário. Uma implantação que esqueça de informar a variável passa a rodar a aplicação como superusuário do banco em vez do papel dedicado — o que significa que uma injeção de SQL, um <code>pg_read_file</code> ou qualquer defeito de escrita deixa de ser 'estrago dentro do schema da aplicação' e vira 'estrago no cluster inteiro'. Impacto: Amplia o raio de qualquer outra falha. Não é explorável por si só — é a diferença entre um incidente contido e um incidente total. Condição de explorabilidade: Não ocorre nas configurações atuais:

Severidade	Arquivo:linha	Descrição
		`env.docker.example` e `env.vps.example` definem `POSTGRES_USER=controle_bancario`. O achado é sobre a escolha do default e sobre não haver validação que recuse `postgres` como usuário da aplicação. <pre>POSTGRES_USER: \${POSTGRES_USER:-postgres}</pre> CORRIGIDO — o que mudou: O default de `POSTGRES_USER` nos três pontos do `compose.yaml` (linhas 22, 78, 88) mudou de `postgres` para `controle_bancario`. `financeiro/settings.py` ganhou uma checagem de boot logo depois de montar `DATABASES`: se `DATABASES['default']['USER'] == 'postgres'`, levanta `RuntimeError` nomeando a variável — mesma família de validação que `USE_HTTPS`/`CSRF_TRUSTED_ORIGINS` já tinha. `env.docker(.example)` e `env.vps.example` já declaravam `POSTGRES_USER=controle_bancario` explicitamente, então nenhuma implantação real muda de comportamento — o que fecha é o default silencioso, não a configuração em uso. Verificação: tests/test_operational_configuration.py (novos): test_settings_recusa_conectar_como_postgres, test_settings_aceita_postgres_user_dedicado, test_default_de_postgres_user_nao_e_o_superuser (importa financeiro.settings sem a variável definida e confere que DATABASES['default']['USER'] == 'controle_bancario') e test_compose_nao_usa_postgres_como_padrao_de_postgres_user.

5. Inputs sem tratamento

Severidade	Arquivo:linha	Descrição
INFORMATIVA CORRIGIDO	core/templatetags/icon_s.py:38-45	[CB-06] A templatetag `icon` marca o rótulo como seguro sem escapá-lo `icon(name, label=)` interpola `label` num `<span>` e devolve o resultado por `mark_safe`, sem passar por `escape()`. O SVG vem de um dicionário fechado e é seguro; o `label` é um parâmetro livre de template. Por que é explorável: Hoje não é explorável: a varredura de todos os templates encontrou apenas chamadas da forma `{% icon 'nome' %}`, sem segundo argumento — nenhum call site passa `label`, e muito menos um valor vindo do usuário. O que existe é um contrato que convida ao erro: o dia em que alguém escrever `{% icon 'edit' conta.nome %}`, o nome da conta entra no HTML sem escape e a CSP estrita passa a ser a única linha de defesa. Impacto: Nenhum hoje. Registrado porque a correção custa uma linha e elimina a categoria inteira do arquivo, em vez de depender de ninguém nunca usar o segundo parâmetro. Condição de explorabilidade: Exige que um call site futuro passe `label` com valor controlado pelo usuário. <pre>@register.simple_tag def icon(name: str, label: str = "") -> str: svg = _ICONS.get(name, "") if not svg: return "" if label: return mark_safe(f"{svg}{label}") return mark_safe(svg)</pre> CORRIGIDO — o que mudou: `icon(name, label=)` em `core/templatetags/icons.py` passou a escapar `label` com `django.utils.html.escape` antes de interpolar no `<span>`, dentro do

Severidade	Arquivo:linha	Descrição
		próprio <code>`mark_safe`</code> — fecha a categoria no arquivo, sem depender de nenhum call site futuro lembrar de escapar antes de passar o segundo parâmetro. Verificação: tests/test_icons_templatetag.py (novo): test_label_com_html_e_escapado (prova que <code>`<script>`</code> vira <code>`&lt;script&gt;`</code>), test_label_comum_continua_visivel (regressão do caminho feliz), test_sem_label_nao_acrescenta_span e test_nome_desconhecido_devolve_vazio.

5. Recomendações priorizadas

Prio	Ação	Achados
P1 FEITA	Corrigir <code>`retire_budget`</code> para receber o usuário e exigir <code>`accessible_owner_ids(user, 'delete')`</code> sobre <code>`budget.owner_id`</code> , no mesmo formato que <code>`save_budget`</code> já usa 25 linhas acima. Acrescentar <code>`log_audit_event`</code> na view, que hoje é a única das ações de Gestão sem registro de auditoria.	CB-01
P1 FEITA	Fechar a auto-elevação por <code>`permissions.manage`</code> : quando <code>`selected_user.id == request.user.id`</code> , recusar a concessão de permissão funcional ou de acesso a titular que a própria pessoa ainda não tenha — mantendo a proteção atual, que impede remover a própria <code>`permissions.manage`</code> . Alternativa mais simples e igualmente defensável: exigir <code>`user_type == administrator`</code> para as duas ações, já que administrar permissões alheias é privilégio administrativo por natureza.	CB-02
P3 FEITA	Limpar <code>`POSTGRES_PASSWORD`</code> e <code>`DJANGO_SECRET_KEY`</code> de <code>`.env.docker`</code> depois de provisionar, e conferir se <code>`scripts/package_clean_zip.py`</code> exclui <code>`.env*`, `.secrets/`</code> e <code>`.certs/`</code> . Rotacionar não é necessário: os valores não vazaram.	CB-03
P1 FEITA	Remover <code>`django.contrib.admin`</code> de <code>`INSTALLED_APPS`</code> e a rota <code>`admin/`</code> da URLconf, junto dos oito <code>`admin.py`</code> vazios. Nenhum modelo do projeto está registrado e nenhum fluxo usa o painel; o que resta dele é um caminho de administração de contas sem as travas e sem a auditoria da aplicação, mais uma porta de autenticação que contorna o bloqueio de tentativas. Depois disso, zerar <code>`is_staff`</code> e <code>`is_superuser`</code> da conta <code>`Admin`</code> em produção.	CB-04
P3 FEITA	Trocar o default de <code>`POSTGRES_USER`</code> de <code>`postgres`</code> para o nome dedicado da aplicação, e acrescentar a <code>`settings.py`</code> uma checagem que recuse subir conectando como <code>`postgres`</code> — a mesma família de validação de boot que <code>`USE_HTTPS`/`CSRF_TRUSTED_ORIGINS`</code> já tem.	CB-05
P3 FEITA	Escapar o <code>`label`</code> em <code>`icons.py`</code> com <code>`django.utils.html.escape`</code> antes de interpolar, fechando a categoria no arquivo em vez de depender de ninguém usar o segundo parâmetro.	CB-06

6. Issues para o GitHub

Cada bloco abaixo é o texto COMPLETO de uma issue, em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só, para não gerar ruído no rastreador. As issues já resolvidas continuam aqui, marcadas — apagá-las deixaria o relatório sem o registro do que foi fechado —, mas não devem ser abertas no rastreador.

✓ Issue 1 — [Segurança] IDOR em `retire\_budget`: apaga orçamento de qualquer titular

```
--- ISSUE 1 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] IDOR em `retire_budget`: apaga orçamento de qualquer titular
Labels: security, severity:media, idor, isolamento

## Problema

`retire_budget(budget_id)` recebe apenas o ID: não recebe o usuário e não consulta `accessible_owner_ids`. Ela resolve o `MonthlyBudget`, decide entre arquivar (se há lançamento realizado no mês/categoria/titular) ou apagar de vez, e executa.

A view que a chama exige `management.manage` — a permissão FUNCIONAL — e nada mais.

### Por que é explorável

A assimetria é a prova de que é defeito, e não decisão. `save_budget`, na MESMA camada e a 25 linhas de distância, faz a checagem correta. A listagem da própria tela também está escopada: `management_view` monta `budget_rows_for_period` a partir de `options.owners`, que vem de `context_options` já filtrado.

Ou seja: a pessoa **não vê** o orçamento de outro titular na tela, **não pode criar nem editar** orçamento de outro titular — e **pode apagar** o de qualquer um. Basta um POST direto, com o token CSRF da própria sessão:

...

POST /management/budget/retire/
Content-Type: application/x-www-form-urlencoded

csrfmiddlewaretoken=<token da propria sessao>&budget_id=<id de outro titular>
...

O ID não precisa ser adivinhado com precisão: é uma chave sequencial, e a resposta distingue "Orçamento não encontrado" de sucesso.

## Evidência

**O defeito** — `management/services.py:225-244`

```python
def retire_budget(budget_id) -> tuple[MonthlyBudget, str]:
 try:
 budget = MonthlyBudget.objects.select_related("owner",
 "category").get(id=int(budget_id))
 except (MonthlyBudget.DoesNotExist, TypeError, ValueError):
 raise ValueError("Orçamento não encontrado.") from None

 start, end_exclusive = month_bounds(budget.year, budget.month)
 has_history = CashFlowEntry.objects.filter(...).exists()
 if has_history:
 budget.active = False
 budget.save(update_fields=["active", "updated_at"])
 return budget, "archived"
 budget.delete()
 return budget, "deleted"
...

O padrão correto, na mesma camada — `management/services.py:200-201`

```python
if owner_id_int not in set(accessible_owner_ids(user, "update")):
    raise ValueError("Acesso negado: voce nao pode gerenciar orcamento deste titular.")
...

**A view** — `management/views.py:117-122`

```python
@login_required
@permission_required("management.manage", fallback="management:management_view")
@require_POST
def retire_budget_view(request):
 try:

```

```

 budget, action = services.retire_budget(request.POST.get("budget_id"))
 ...

Impacto

Escrita destrutiva atravessando a fronteira de titular, que é o mecanismo de isolamento deste sistema. O dado perdido é de planejamento (valor orçado por categoria/mês), não do razão – o que limita o estrago –, mas a exclusão é definitiva quando não há lançamento realizado no período, e não há tela que a desfaça.

O evento ``também não entra na trilha de auditoria``: `retire_budget_view` é a única ação de Gestão que não chama `log_audit_event`.

``Condição de explorabilidade`` conta autenticada com `management.manage`, e mais de um titular no sistema com acessos distintos. Para `administrator` e `super_user` o comportamento é o mesmo com ou sem o defeito, porque `_has_broad_owner_access` já lhes dá escopo amplo – o alvo é a conta de tipo `user` com escopo restrito.

Sugestão de correção

```python
def retire_budget(user, budget_id) -> tuple[MonthlyBudget, str]:
    try:
        budget = MonthlyBudget.objects.select_related("owner",
            "category").get(id=int(budget_id))
    except (MonthlyBudget.DoesNotExist, TypeError, ValueError):
        raise ValueError("Orçamento nao encontrado.") from None

    # Mesmo escopo de `save_budget`: `delete` e a acao correspondente a esta.
    if budget.owner_id not in set(accessible_owner_ids(user, "delete")):
        raise ValueError("Acesso negado: voce nao pode remover orcamento deste titular.")
    ...
    ...

E na view, o registro que falta:

```python
 budget, action = services.retire_budget(request.user, request.POST.get("budget_id"))
 log_audit_event("monthly_budget", budget.id, action, user=request.user)
...

Critérios de aceite

- [] `retire_budget` recebe o usuário e recusa titular fora de `accessible_owner_ids(user, "delete")`
- [] A mensagem de recusa não revela se o orçamento existe
- [] Teste: usuário com acesso ao titular A recebe recusa ao remover orçamento do titular B
- [] Teste: usuário com acesso ao titular A continua removendo orçamento de A
- [] Teste: `administrator` continua removendo qualquer orçamento
- [] `retire_budget_view` registra `log_audit_event` no sucesso
- [] Uma varredura da camada `management/services.py` confirma que nenhuma outra função de escrita ficou sem escopo
--- FIM ISSUE 1 ---

```

## ✓ Issue 2 — [Segurança] ``permissions.manage`` permite auto-concessão de todo o escopo de dados

```

--- ISSUE 2 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] `permissions.manage` permite auto-concessão de todo o escopo de dados
Labels: security, severity:media, escalonamento, autorizacao

Problema

A tela de Permissões exige apenas `permissions.manage`. O usuário-alvo vem de `request.GET/POST['user_id']`, resolvido sobre `list_manageable_users()`, que devolve TODOS os usuários – inclusive quem está operando.

Duas ações dessa tela não têm trava de auto-elevação:

- `save_function_permissions` grava o conjunto inteiro de permissões funcionais no alvo;
- `save_owner_access` grava a matriz completa de view/create/update/delete sobre ``todos`` os titulares.

Por que é explorável

A única proteção contra manipular a própria conta é o oposto do que seria preciso: impede REMOVER a própria `permissions.manage`, e não ACRESCENTAR o resto.

Um usuário de tipo `user`, com escopo restrito a um titular e com `permissions.manage` concedida:

```

1. abre `/permissions/?user\_id=<o proprio id>`;
2. marca todas as caixas de permissão funcional, salva;
3. marca todas as linhas da matriz de titulares, salva;
4. passa a ver e alterar todo o dado financeiro do sistema.

O contraste mostra que a preocupação com escalonamento existe e parou no lugar errado: `user\_mutation\_block\_message` bloqueia explicitamente a promoção de `user\_type` por quem não é administrador; nada equivalente cobre o escopo de dados.

## Evidência

`core/views.py:102-105`

```
```python
@login_required
@permission_required('permissions.manage')
def permissions_view(request):
    users = list_manageable_users()
    selected_user_id = request.GET.get('user_id') or request.POST.get('user_id')
    selected_user = users.filter(id=selected_user_id).first() if selected_user_id else
    users.first()
...`
```

`core/views.py:211-220` – a "proteção", que só impede a remoção:

```
```python
if action == 'save_function_permissions':
 allowed_keys = {
 key for key in PERMISSION_DEFINITIONS
 if request.POST.get(f'permission_{key}') == 'on'
 }
 if selected_user.id == request.user.id and 'permissions.manage' not in allowed_keys:
 allowed_keys.add('permissions.manage')
 messages.warning(request, "Protecao aplicada: voce nao pode remover o proprio
acesso...")
 save_function_permissions(selected_user, allowed_keys)
...`
```

`core/views.py:228-241` – a matriz de titulares, sem trava alguma:

```
```python
elif action == 'save_owner_access':
    owner_flags: dict[int, dict[str, bool]] = {}
    for owner in AccountOwner.objects.all():
        owner_flags[owner.id] = {
            'view': request.POST.get(f'owner_{owner.id}_view') == 'on',
            ...
        }
    save_owner_access_matrix(selected_user, owner_flags)
...`
```

`accounts/services.py:356-360` – a trava que existe, para `user\_type`:

```
```python
if action in ("add", "edit") and _is_elevated_user_type(requested_user_type) \
 and current_user.user_type != USER_TYPE_ADMINISTRATOR:
 return "Acesso negado: somente administrator pode criar ou promover usuarios
privilegiados."
...`
```

## Impacto

Escalonamento de privilégio dentro do modelo de permissões: acesso completo aos dados de todos os titulares e a todas as funções, sem passar por `tables.users.manage` e sem virar `administrator`.

A ação **\*\*fica registrada\*\*** na trilha de auditoria – `log\_audit\_event` é chamado nas duas –, o que ajuda a detectar, mas não impede.

**\*\*Condição:\*\*** exige que `permissions.manage` tenha sido concedida a uma conta não administradora. Se ela só existe em contas `administrator`, não há escalonamento (o administrador já tem tudo por regra). O risco nasce no momento em que alguém delega a gestão de permissões sem delegar o resto – que é exatamente o caso de uso que a permissão existe para atender.

## Sugestão de correção

Duas saídas, ambas defensáveis. A primeira preserva a delegação:

```
```python
def _recusar_auto_elevacao(request, selected_user, allowed_keys):
    """Ninguem concede a si mesmo o que ainda nao tem."""
    if selected_user.id != request.user.id:
        return allowed_keys
    atuais = allowed_permission_keys(request.user)
    return {k for k in allowed_keys if k in atuais} | {'permissions.manage'}```
```

```

...

e o equivalente para `owner_flags`, cortando pela interseção com
`owner_access_map(request.user)`.

A segunda é mais simples e igualmente defensável: exigir
`user_type == administrator` para as duas ações, já que administrar permissões
alheias é privilégio administrativo por natureza. Nesse caso
`permissions.manage` passa a ser, na prática, uma permissão de LEITURA da tela.

## Critérios de aceite

- [ ] Um usuário não administrador com `permissions.manage` não consegue conceder a si mesmo
  permissão funcional que já não tenha
- [ ] O mesmo usuário não consegue conceder a si mesmo acesso a titular que já não tenha
- [ ] A proteção atual (não remover a própria `permissions.manage`) continua valendo
- [ ] `administrator` continua gerindo tudo, inclusive a própria conta
- [ ] Teste por caso: auto-concessão de permissão funcional, auto-concessão de titular, e os
  dois caminhos para administrador
- [ ] A recusa aparece como mensagem na tela, não como erro 500
- [ ] `docs/architecture.md` registra o que `permissions.manage` passa a significar
--- FIM ISSUE 2 ---

```

✓ Issue 3 — [Segurança] Segredos duplicados em texto claro em `.env.docker`

```

--- ISSUE 3 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] Segredos duplicados em texto claro em `.env.docker`
Labels: security, severity:baixa, segredos, higiene

## Problema

`.env.docker` guarda `POSTGRES_PASSWORD` e `DJANGO_SECRET_KEY` em texto claro –
os mesmos valores que já estão em `.secrets/postgres_password` e
`.secrets/django_secret_key`, confirmado por comparação de hash.

A duplicação é o desenho do fluxo, não um acidente:
`scripts/provision_compose_secrets.ps1:39-43` **copia** do `.env` para
`.secrets/`, e falha se a variável não estiver lá.

### O que NÃO é

Não houve exposição. O que foi verificado:

- `.gitignore:35` recusa `.env.*`, confirmado por `git check-ignore`;
- o histórico completo do Git não tem nenhum arquivo de segredo;
- `.dropboxignore:38-39` já excluía `.env` e `.env.*` da sincronização de nuvem
  ANTES desta auditoria – conferido em `git show HEAD:.dropboxignore`;
- a pasta `.secrets/` foi verificada em dropbox.com, inclusive na lixeira:
  nunca subiu.

E este projeto tem a mitigação mais forte dos quatro: `REQUIRE_FILE_SECRETS`
faz o contêiner **recusar** a forma direta por variável.

### Por que ainda vale corrigir

Superfície. O mesmo segredo existe em dois arquivos, e a `DJANGO_SECRET_KEY`
assina três coisas aqui: cookie de sessão, token CSRF e o token de confirmação
de "este e os próximos" de `transactions/services.py`.

E há um detalhe específico deste repositório: existe um
`scripts/package_clean_zip.py`. Ferramenta de empacotamento é exatamente onde
um arquivo a mais na raiz costuma entrar sem ninguém notar – vale conferir a
lista de exclusão dele.

## Evidência

`financeiro/settings.py:29-34` – a mitigação que já existe:

```python
require_file = os.environ.get("REQUIRE_FILE_SECRETS", "false").lower() == "true"
return resolver_segredo(
 name,
 aceitar_variavel=not require_file,
 obrigatorio=True,
)
```

`compose.yaml:25-26`:

```yaml
DJANGO_SECRET_KEY_FILE: /run/secrets/django_secret_key
REQUIRE_FILE_SECRETS: "true"
```

## Impacto

```

```

Nenhum comprometimento conhecido. Custo de higiene e de raio de alcance.

## Sugestão de correção

O fluxo atual exige o valor no `.env` para provisionar. Duas saídas:

1. Mínima: depois de rodar `provision_compose_secrets.ps1`, limpar as duas
  linhas do `.env.docker` – o Compose não precisa delas, e o `Force` só é
  necessário numa rotação deliberada;
2. Melhor: fazer o script GERAR os valores quando ausentes (como o do
  MegaSena faz) em vez de exigir que estejam no `.env`, e nunca gravá-los lá.

Conferir também `scripts/package_clean_zip.py`: `.env*`, `.secrets/` e
`.certs/` devem estar na exclusão.

Rotacionar não é necessário: os valores não vazaram.

## Critérios de aceite

- [ ] `.env.docker` não contém mais `POSTGRES_PASSWORD` nem `DJANGO_SECRET_KEY`
- [ ] A pilha sobe normalmente com os segredos vindos de `/run/secrets`
- [ ] `REQUIRE_FILE_SECRETS=true` continua no `compose.yaml` (regressão: é ele que recusa a
  forma direta)
- [ ] `scripts/package_clean_zip.py` exclui `.env*`, `.secrets/` e `.certs/`, com teste
- [ ] `.dropboxignore` mantém `.env.*`, `.secrets/` e `.certs/` (regressão)
--- FIM ISSUE 3 ---

```

✓ Issue 4 — [Segurança] `/admin/` do Django administra contas por fora das travas e do bloqueio de tentativas

```

--- ISSUE 4 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] `/admin/` do Django administra contas por fora das travas e do bloqueio de
tentativas
Labels: security, severity:media, autenticacao, autorizacao

## Problema

`django.contrib.admin` está em `INSTALLED_APPS` e `path('admin/', admin.site.urls)`
está roteado.

Nenhum `admin.py` do projeto registra modelo – os oito são stubs vazios do
`startapp`. Mas `django.contrib.auth.admin` registra o modelo de usuário
sozinho, e o modelo registrado é o substituído: `AppUser`.

### Por que é explorável

A consulta ao banco de produção em 01/09/2026 fecha a questão:
...

mspa      tipo=administrator  staff=False super=False    <- conta de trabalho
Claudia   tipo=user                 staff=False super=False
Esther    tipo=user                 staff=False super=False
Admin     tipo=administrator  staff=True  super=True    <- alcanca /admin/
...

São dois problemas distintos:

**1. Administração de contas por fora das travas.** Com a conta `Admin`, o
`/admin/` edita `AppUser` diretamente – criar conta, trocar senha, alterar
`user_type`, ligar `is_superuser`. Tudo por fora de
`user_mutation_block_message` (que impede promover usuário privilegiado sem ser
administrador), por fora da regra do último administrador, por fora da
obrigação de trocar senha temporária, e sem registrar nada em
`log_audit_event`.

**2. O bloqueio de tentativas é contornado.** `AdminAuthenticationForm` chama
`authenticate()` – e portanto verifica a senha – antes de recusar por falta de
`is_staff`. Um POST em `/admin/login/` não passa por
`assert_login_not_throttled`, não chama `register_failed_login_attempt` e não
tem limite de taxa.

## Evidência

`financeiro/settings.py:60-61` e `financeiro/urls.py:76`

```python
INSTALLED_APPS = [
 'django.contrib.admin',
 ...
 path('admin/', admin.site.urls),
 ...

`accounts/views.py:107-114` – onde o bloqueio existe, e só ali:

```python
class AppLoginView(LoginView):

```

```

def post(self, request, *args, **kwargs):
    remote_addr = request.META.get('REMOTE_ADDR')
    try:
        assert_login_not_throttled(request.POST.get('username'), remote_addr)
    except LoginThrottledError as exc:
        messages.error(request, str(exc))
        return self.render_to_response(self.get_context_data())
    return super().post(request, *args, **kwargs)
...

## Impacto

Um caminho paralelo de administração de contas, sem as travas da aplicação e
sem trilha de auditoria, disponível a quem obtiver a senha da conta `Admin`.
Somado a isso, adivinhação de senha ilimitada contra qualquer conta conhecida.

O sistema tem quatro contas, duas delas do tipo `user` – não é cenário de
usuário único.

## Sugestão de correção

Remover o que não é usado. `django.contrib.auth` e `django.contrib.messages`
continuam necessários e não dependem do admin:

```python
financeiro/settings.py
INSTALLED_APPS = [
 # 'django.contrib.admin', <- nenhum modelo do projeto registrado
 'django.contrib.auth',
 ...

financeiro/urls.py
... # path('admin/', admin.site.urls),
...

Os oito `admin.py` vazios saem junto: sem o app instalado eles nunca são
importados, e um arquivo chamado `admin.py` sugere ao próximo leitor um painel
que não existe.

Depois disso, tirar as marcas da conta `Admin` em produção – com o app
removido elas ficam inertes, mas continuam sendo privilégio declarado sem uso:

```python
AppUser.objects.filter(username="Admin").update(is_staff=False, is_superuser=False)
```

Se a conta `Admin` não for usada por ninguém (o último login é anterior ao da
conta de trabalho), avaliar desativá-la – a regra do último administrador
protege contra o auto-lockout, e `mspa` já é `administrator`.

Critérios de aceite

- [] `GET /admin/` e `POST /admin/login/` respondem **404** (e não 302: um 302 significaria que
 a URLconf ainda casa o caminho)
- [] A aplicação sobe e a suíte passa sem `django.contrib.admin`
- [] `collectstatic` continua funcionando (o build da imagem o executa)
- [] Nenhum template referencia `admin/` (verificado por varredura)
- [] Os oito `admin.py` vazios foram removidos
- [] `tests/test_permissaoes_por_rota.py` deixa de abrir exceção para `admin/` e volta a varrer
 a URLconf inteira
- [] `is_staff` e `is_superuser` da conta `Admin` foram zerados em produção
- [] `/login` continua sendo a única rota que verifica senha, e o `LoginLockout` continua
 funcionando
--- FIM ISSUE 4 ---

```

## ✓ Issue 5 — [Segurança] Defaults do Compose e escape do rótulo em `icons.py`

```

--- ISSUE 5 ---
*** JÁ RESOLVIDA -- NÃO ABRIR NO RASTREADOR ***

Título: [Segurança] Defaults do Compose e escape do rótulo em `icons.py`
Labels: security, severity:baixa, endurecimento

Problema

Dois pontos pequenos e independentes, agrupados numa issue só porque ambos são
de endurecimento e cada um cabe em poucas linhas.

1. `POSTGRES_USER` cai no superusuário do cluster

Os três pontos do Compose que nomeiam o usuário do banco usam
`${POSTGRES_USER:postgres}`. Ausente a variável, o PostgreSQL é inicializado
com o papel `postgres` – o superusuário – e a aplicação conecta com ele.

O default escolhido é o mais privilegiado possível, e não o mínimo necessário.
Uma implantação que esqueça a variável passa a rodar a aplicação como
superusuário do banco: uma injeção de SQL, um `pg_read_file` ou qualquer defeito

```

```

de escrita deixa de ser "estrago dentro do schema da aplicação" e vira "estrago
no cluster inteiro".

Condição: não ocorre nas configurações atuais — `.env.docker.example` e
`.env.vps.example` definem `POSTGRES_USER=controle_bancario`. O achado é sobre a
escolha do default e sobre não haver validação que recuse `postgres`.

2. `icon()` marca o rótulo como seguro sem escapá-lo

`icon(name, label='')` interpola `label` num `` e devolve o resultado por
`mark_safe`, sem passar por `escape()`.

Hoje **não é explorável**: a varredura de todos os templates encontrou apenas
chamadas da forma `{% icon 'nome' %}`, sem segundo argumento. O que existe é um
contrato que convida ao erro — o dia em que alguém escrever
`{% icon 'edit' conta.nome %}`, o nome da conta entra no HTML sem escape, e a
CSP estrita passa a ser a única linha de defesa.

Evidência

`compose.yaml:22`, `:78` e `:88`

```yaml
  POSTGRES_USER: ${POSTGRES_USER:-postgres}
```

`core/templatetags/icons.py:38-45`

```python
@register.simple_tag
def icon(name: str, label: str = "") -> str:
    """Renderiza o SVG do icone `name`; string vazia se o nome nao existir."""
    svg = _ICONS.get(name, "")
    if not svg:
        return ""
    if label:
        return mark_safe(f"{svg}<span>{label}</span>")
    return mark_safe(svg)
```

Impacto

Nenhum dos dois é explorável na configuração e no código atuais. O primeiro
amplia o raio de qualquer outra falha; o segundo deixa uma armadilha para quem
usar o segundo parâmetro no futuro.

Sugestão de correção

1. Trocar o default e recusar `postgres` no boot, na mesma família de
validação que `USE_HTTPS`/`CSRF_TRUSTED_ORIGINS` já tem:

```yaml
  POSTGRES_USER: ${POSTGRES_USER:-controle_bancario}
```

```python
# financeiro/settings.py
if DATABASES['default']['USER'] == 'postgres':
    raise RuntimeError(
        'A aplicacao nao deve conectar como superusuario do cluster. '
        'Defina POSTGRES_USER com o papel dedicado.'
    )
```

2. Escapar o rótulo:

```python
from django.utils.html import escape

...
    if label:
        return mark_safe(f"{svg}<span>{escape(label)}</span>")
...
```

Critérios de aceite

- [] O default de `POSTGRES_USER` no `compose.yaml` não é mais `postgres`, nos três pontos
- [] Subir com `POSTGRES_USER=postgres` levanta erro nomeando a variável
- [] A pilha sobe normalmente com o papel dedicado (teste manual de `docker compose up`)
- [] `icon()` escapa o `label` antes de interpolar
- [] Teste: `icon('edit', '<script>x</script>')` devolve o texto escapado, não a tag
- [] Teste: `icon('edit')` continua devolvendo o SVG intacto
- [] Todas as telas continuam renderizando os ícones (verificado na tela)
--- FIM ISSUE 5 ---

```